UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias
Licenciatura en Ciencias de la Computación

Asignatura: Inteligencia Artificial

Laboratorio 2.1: Introducción a Python y Entornos de Trabajo
Temas: El Zen de Python, Tipos de Datos y Operadores

Profesora: Dra. Jessica Sarahi Méndez Rincón
Estudiante: Sita Zoé Sotomayor Carrillo
Fecha: 3 de marzo de 2026

1. Introducción

El presente reporte describe las actividades realizadas en el Laboratorio 2.1 de la asignatura de Inteligencia Artificial. El objetivo principal de esta práctica fue familiarizarse con el entorno de desarrollo Google Colab y repasar los fundamentos del lenguaje de programación Python. Se exploraron conceptos esenciales como la filosofía del lenguaje (Zen de Python), la asignación de variables, los tipos de datos primitivos y las operaciones aritméticas, elementos base para la construcción de algoritmos de IA.

2. Desarrollo

La práctica se estructuró en tres secciones principales dentro del cuaderno de Jupyter (`.ipynb`). A continuación se describe la metodología y el código implementado en cada una.

2.1. El Zen de Python y la Filosofía del Diseño

Se inició la práctica ejecutando el comando `import this`, el cual despliega los principios rectores del diseño de Python. Como parte de la actividad, se seleccionaron frases clave que resaltan la legibilidad y simplicidad del código, almacenándolas en una variable de tipo cadena.

```python
zen_fragmento = """
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
"""
print(zen_fragmento)
```

Listing 1: Ejemplo de almacenamiento de texto

2.2. Tipos de Datos y Variables

Se realizaron ejercicios para identificar y manipular los tipos de datos fundamentales en Python. Se verificó el comportamiento dinámico del lenguaje asignando valores a variables y comprobando su tipo con la función `type()`.

- **Cadenas (str):** Se utilizaron comillas simples, dobles y triples.
- **Enteros (int) y Flotantes (float):** Se observó cómo Python diferencia números con y sin punto decimal.
- **Booleanos (bool):** Se asignaron valores de verdad (`True`, `False`).

2.3. Calculadora de Tiempo (Operadores Aritméticos)

La actividad final consistió en programar una calculadora que convierte unidades de tiempo. Se solicitó al usuario ingresar una cantidad de segundos para realizar conversiones.

```python
# Solicitamos al usuario que ingrese los segundos en una hora
segundos_por_hora = float(input("Ingrese segundos en una hora: "))
segundos_por_dia = segundos_por_hora * 24

# Realizamos las divisiones solicitadas
division_flotante = segundos_por_dia / segundos_por_hora
division_entera = segundos_por_dia // segundos_por_hora

print(f"Segundos por d a: {segundos_por_dia}")
print(f"Divisi n flotante: {division_flotante}")
print(f"Divisi n entera: {division_entera}")
```

Listing 2: Código implementado para el cálculo de segundos

3. Análisis de Resultados

De acuerdo con lo solicitado en las instrucciones de la práctica, se analizó el comportamiento de los operadores de división al procesar los datos de tiempo.

Al dividir los segundos totales de un día entre los segundos de una hora, se obtuvieron dos resultados distintos dependiendo del operador:

1. **Operador `/`:** Devolvió un resultado de tipo `float` (ej. `24.0`). Este operador realiza una división real, conservando la precisión decimal.

2. **Operador `//`:** Devolvió un resultado de tipo `int` (ej. `24`). Este operador realiza una división de piso (*floor division*), truncando cualquier parte decimal.

Reflexión: En el contexto de la Inteligencia Artificial, esta distinción es crítica. Mientras que el cálculo de probabilidades o pesos en una red neuronal requiere la precisión de los flotantes (`/`), la manipulación de índices de matrices o contadores de iteraciones requiere estrictamente números enteros (`//`).

4. Conclusión

Esta práctica permitió consolidar los conocimientos sobre la sintaxis básica de Python. La capacidad de Python para manejar tipos de datos de forma dinámica, sumada a la claridad de su sintaxis (como se vio en el Zen de Python), facilita el prototipado rápido de soluciones. El uso de Google Colab demostró ser eficiente para documentar y ejecutar código en bloques, una metodología estándar en la ciencia de datos.

5. Bibliografía

- Matthes, E. (2019). *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*. No Starch Press.

- Lubanovic, B. (2019). *Introducing Python: Modern Computing in Simple Packages*. O'Reilly Media.